JAVA INTERVIEW PREPARATION

CHAPTER 59

HIGH CPU INCIDENT DIAGNOSIS

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

Diagnosing High CPU in Java Services

Senior / Lead Java Developer Reading Chapter

High CPU is a symptom, not a root cause. It may indicate useful work at peak demand, an inefficient algorithm, excessive allocation and garbage collection, retry amplification, a tight loop, lock spinning, serialization, encryption, or another process on the same host. The investigation must connect operating-system consumption to Java execution and then to a business workload.

A senior incident response preserves evidence before changing the system. Restarting may restore service, but it destroys thread state, profiler samples, compiler history, and the workload conditions that reveal the defect.

```
alert -> confirm scope -> protect service -> capture evidence
      -> identify hot process/thread -> identify hot stack
      -> correlate with traffic/change -> mitigate -> correct and verify
```

Establish whether CPU is actually the bottleneck

CPU utilization alone is ambiguous. Eighty percent of one core on a sixteen-core host is very different from eighty percent of the host. Container dashboards may report CPU as cores consumed, a percentage of the container limit, or a percentage of node capacity. Confirm the unit.

Inspect:

- process CPU and host CPU;
- user time versus kernel/system time;
- container CPU limit and throttled time;
- runnable thread count and load average;
- request rate, completed throughput, error rate, and latency;
- garbage-collection CPU and pause activity;
- deployment, traffic, feature, and dependency changes.

If CPU rises with throughput while latency and errors remain healthy, the service may simply be doing more useful work. The question becomes capacity and cost, not incident diagnosis. If CPU is saturated while throughput falls and queues grow, work is being wasted or the service has reached its efficient limit.

Stabilize without erasing the evidence

Protect customers first, but prefer reversible actions. Reduce nonessential traffic, stop a runaway batch job, disable an expensive feature, rate-limit abusive clients, or add already-tested capacity. Avoid sending retries toward a saturated service.

Before restarting the only affected instance, capture at least:

```
timestamp and deployment version
traffic and error snapshot
process and container CPU
three thread dumps several seconds apart
short JFR or sampling profile
GC and memory state
```

One thread dump is a photograph. Repeated dumps reveal a thread remaining in the same runnable stack, or many threads repeatedly reaching the same method. A short profile provides statistical evidence about where

CPU time accumulates.

Do not enable an unfamiliar high-overhead profiler across the entire fleet during an incident. Start with Java Flight Recorder or a tested sampling profiler on one representative instance and use a bounded duration.

Separate host, container, and JVM views

The host may be busy because another process consumes CPU, the kernel handles network or storage work, or a neighboring container creates contention. A JVM dashboard cannot establish host ownership by itself.

In a container, CPU throttling can cause high latency even when the service does not appear to consume a full physical host. When the process exhausts its CPU quota, it waits until the next scheduling period. This resembles an application pause but is imposed outside the JVM.

```
host has spare CPU
container reaches its configured quota
-> throttled periods increase
-> runnable Java work waits
-> latency rises despite moderate host utilization
```

Compare requested and limited CPU with actual use. Very low limits can distort JVM ergonomics and make startup, JIT compilation, GC, and request work compete for a small quota. Raising a limit may mitigate the incident, but the root cause can still be a regression that created more work.

Find the hot Java thread

Operating-system tools report a native thread identifier. Java thread dumps display a native ID, commonly as `nid` in hexadecimal. Convert and match the IDs, taking care that commands and formats vary by platform and JDK.

```
OS reports hot thread: 18437 decimal
convert to hexadecimal: 0x4805
thread dump: "worker-17" ... nid=0x4805 runnable
```

Then compare that thread across multiple dumps. A `RUNNABLE` state means eligible for execution; it does not prove that the thread continuously used CPU. The OS CPU sample and profiler evidence supply that link.

If several threads share CPU, a sampling profiler is more useful than manually mapping one ID. Also inspect kernel time: heavy system CPU may point to networking, file I/O, page faults, native libraries, or profiler-unfriendly work rather than pure Java methods.

Use Java Flight Recorder deliberately

Java Flight Recorder records JVM and application events with production-oriented overhead. If a continuous recording is already running, dump the relevant time window. Otherwise start a bounded recording during the symptom.

```
jcmd <pid> JFR.start name=cpu-incident settings=profile duration=120s filename=cpu.jfr
jcmd <pid> JFR.check
jcmd <pid> JFR.dump name=cpu-incident filename=cpu-now.jfr
```

Exact commands should be validated for the deployed JDK. JFR analysis can connect method samples with thread activity, allocation, garbage collection, monitor contention, socket I/O, exceptions, compilation, and environment information.

Interpret samples statistically. A method appearing at the top of a flame graph means sampled CPU stacks frequently included that frame. It does not necessarily mean the method is defective; it may be the correct

place where legitimate work occurs. Compare with a healthy recording under similar traffic.

CPU profiles versus wall-clock profiles

A CPU profile samples threads while they execute on CPU. It is ideal for algorithms, serialization, compression, parsing, hashing, and garbage-collection-related application work. A wall-clock profile samples elapsed stacks including waiting and sleeping. It is useful for latency and thread-pool diagnosis.

If a request is slow because it waits on a database, the method may dominate a wall profile but consume little CPU. If a regex loop burns a core, it dominates the CPU profile. Use the profile that matches the question.

Async-profiler can sample CPU, allocation, locks, and wall time, subject to operating-system permissions and deployment validation. Flame graphs aggregate stacks:

```
width  = how often a stack path was sampled
height = call depth, not time
color  = presentation category, not severity unless configured
```

Do not conclude that the widest leaf alone is the root cause. Read downward to the business entry point and compare the stack with traffic and code changes.

Common cause: algorithmic amplification

An operation that was harmless for 100 elements can dominate CPU for 100,000. Nested scans, repeated sorting, accidental quadratic string building, and searching a list inside a loop are common.

```
// O(customers * orders): repeated scan
for (Customer customer : customers) {
    long count = orders.stream()
            .filter(o -> o.customerId().equals(customer.id()))
            .count();
    totals.put(customer.id(), count);
}

// Build one index, then perform near-constant-time lookups.
Map<String, Long> counts = orders.stream().collect(
        Collectors.groupingBy(Order::customerId, Collectors.counting()));
```

The optimized version uses more retained memory for the index. Senior reasoning compares CPU, memory, data size, and reuse rather than treating Big-O as the only cost.

Profiles should be segmented by endpoint or task. A method can be efficient per invocation but called millions of times because a cache key changed, pagination broke, or retries multiplied the workload.

Common cause: allocation and garbage collection

High allocation rate consumes CPU in constructors, copying, zeroing memory, and garbage collection. The heap may not grow because objects die quickly; memory usage charts can look stable while CPU is wasted.

Look for allocation rate, young-GC frequency, GC CPU percentage, promotion, and surviving object patterns. JFR allocation samples or an allocation profile can identify sources such as:

- repeated JSON trees and temporary byte arrays;
- logging argument construction even when a level is disabled;
- collecting streams into intermediate lists;
- copying large payloads between layers;
- regex matchers and string transformations;

- boxing in numeric hot paths.

Do not respond by increasing heap automatically. A larger heap may reduce collection frequency but does not remove allocation CPU and can change pause or footprint behavior. Eliminate unnecessary allocation, stream payloads, reuse immutable metadata safely, and benchmark the complete path.

Common cause: garbage-collector pressure

GC threads consume process CPU. Determine whether high CPU is primarily application threads or collector threads. Frequent young collections may follow an allocation surge. Concurrent marking may rise because the live set grew. Repeated full collections with little reclaimed memory indicate memory pressure and can combine high CPU with long pauses.

Correlate GC logs or JFR events with allocation and request traffic. A collector change is rarely the first fix for a new application regression. Compare heap occupancy after collection, allocation rate, live-set trend, pause time, and GC CPU.

When the live set nearly fills the heap, the JVM may spend increasing effort finding little reclaimable space. Treat this as a memory investigation as well as a CPU incident.

Common cause: retry and error storms

A slow dependency can trigger timeouts and retries. Each retry repeats authentication, serialization, logging, tracing, connection acquisition, and exception construction. Multiple layers retrying can multiply calls geometrically.

```
gateway retries 3 times
service retries dependency 3 times
one user request can create up to 9 dependency attempts
```

Exception-heavy paths are expensive because stack traces, logs, and telemetry consume CPU and allocation. Measure attempts per original request, timeout rate, exception rate, and log volume. Stop retrying nontransient errors, cap total attempts, add jitter, and respect a shared deadline and retry budget.

Common cause: parsing, regex, and serialization

Large JSON payloads, deep object graphs, schema conversions, compression, encryption, and signatures are legitimate CPU work. Unexpected payload growth or an algorithm change can saturate cores.

Pathological regular expressions can backtrack excessively on crafted or unusual input. Capture the hot stack and representative input safely. Prefer bounded input size and linear-time parsing where possible. Do not log sensitive production payloads merely to reproduce the issue.

Serialization profiles often point to reflection, property discovery, character encoding, or buffer copying. Cache thread-safe metadata rather than serialized business values, avoid converting between formats repeatedly, and stream large responses when the contract permits it.

Common cause: contention and spinning

Blocked threads normally do not consume much CPU, but spin locks, aggressive polling, failed compare-and-set loops, and zero-delay retry loops can. A thread may repeatedly check shared state without useful progress.

```
// Dangerous busy wait
while (!ready.get()) {
    // consumes a core
}

// Coordination primitive allows the scheduler to park the waiter.
readyLatch.await();
```

Some short spinning is deliberate in low-latency algorithms. It becomes harmful when waits are unpredictable, containers have limited cores, or many threads spin together. Lock profiles, CPU stacks, and source inspection reveal the distinction.

Common cause: JIT compilation and deoptimization

After startup or a deployment, compiler threads may consume CPU while hot code is optimized. This should settle. Repeated class generation, unstable call profiles, or deoptimization can sustain compilation work.

Compare the incident with JVM uptime and deployment time. A cold fleet receiving full traffic immediately may combine class loading, cache misses, connection creation, and JIT compilation. Use readiness, gradual traffic ramp, and realistic warm-up rather than assuming autoscaled instances instantly provide steady-state capacity.

Generated proxy classes or dynamic scripting can also grow code cache or metaspace. JFR compiler and class-loading events help distinguish this from normal warm-up.

Virtual threads do not create CPU

Virtual threads make blocking concurrency cheaper; they do not make CPU-bound work faster. Thousands of runnable virtual threads still compete for the available carrier CPU. A traffic burst can therefore create high runnable concurrency, allocation, and scheduler work even though the application avoids a traditional fixed thread-pool queue.

Bound CPU-intensive sections with semaphores, executors sized for compute, or admission control. Separate blocking I/O concurrency from CPU capacity.

Correlate with the business dimension

A profile tells where CPU is spent. Metrics and traces explain why that path became frequent. Compare by normalized endpoint, tenant class, payload band, result status, cache hit/miss, and deployment version while avoiding unbounded labels.

Useful questions include:

- Did request rate increase, or did CPU per completed request increase?
- Is one tenant, key, or payload shape dominant?
- Did cache misses, retries, exceptions, or response size change?
- Is the regression present only on the new version?
- Does a canary with the previous version show the same profile?

CPU seconds per successful business operation is often more informative than utilization alone.

Fleet-wide versus single-instance CPU

The shape of the incident narrows the hypothesis. If every instance of one version rises together, suspect traffic mix, shared dependency behavior, configuration, or code regression. If one instance is hot, inspect partition ownership, a hot tenant, stuck local state, uneven connection routing, JIT state, or a task assigned only to that node.

```
all instances, same time    -> global workload/configuration/dependency
only new-version instances  -> deployment regression
one instance                -> skew, local loop, ownership, or host condition
one zone                    -> routing, infrastructure, or regional dependency
periodic across fleet       -> scheduled work, expiry wave, or coordinated refresh
```

Use per-instance distributions rather than only fleet averages. One saturated pod can disappear inside a healthy average while its requests time out. Compare the hot instance with a healthy peer using the same traffic class and version.

Partitioned consumers and schedulers deserve special attention. A Kafka partition with a hot key, a shard leader, or a singleton cleanup job can legitimately concentrate work. The fix may be repartitioning, balanced ownership, or bounding that task rather than optimizing the request path.

Profiler safety and interpretation limits

Sampling is safer than instrumenting every method entry and exit in a production incident, but it still needs validation. Short recordings can miss intermittent work; very low sample frequency can hide narrow hot methods; missing native stack support can attribute time to a boundary rather than the true native function.

Capture the profiler command, tool and JDK version, duration, sample event, and target instance with the artifact. Without this metadata, later reviewers may compare incompatible profiles.

CPU samples are biased toward work that remains on CPU long enough to be sampled. Extremely frequent short operations may appear mainly through their callers. Inlined Java methods can be represented differently from source-level expectations, and optimized frames may change after deoptimization.

Validate a proposed optimization with a benchmark or canary. A flame graph is evidence for a hypothesis, not proof of how much end-to-end CPU a source edit will save.

Kernel CPU and native boundaries

High system CPU shifts attention toward syscalls, network processing, filesystem activity, page faults, container runtime overhead, or native libraries. Java stacks may stop at `SocketDispatcher`, compression, cryptography, or JNI boundaries.

Correlate system CPU with packet rate, context switches, disk I/O, logging volume, TLS handshakes, and connection churn. Recreating HTTP clients or database connections can move substantial work into DNS, sockets, TLS, and kernel scheduling. The application cause remains Java ownership even when the consumed cycles appear below the JVM boundary.

Native profiling and kernel events require additional permission and expertise. Follow production security controls and avoid deploying unreviewed privileged tooling during an incident.

Mitigation and permanent correction

Mitigation may include shedding optional work, reverting a deployment, disabling a feature, limiting an abusive workload, scaling proven capacity, pausing batch processing, or reducing retry amplification. Record what was changed and preserve at least one artifact from before mitigation.

The permanent fix should be verified with a representative benchmark and production canary. Compare throughput, p95 and p99 latency, CPU per operation, allocation rate, GC CPU, and correctness. Optimizing one hot method can move the bottleneck to the database or network.

Add regression protection: a performance test, payload limit, complexity guard, cache metric, retry budget, or alert on CPU per operation. A root-cause review should explain why existing tests and observability did not expose the change earlier.

Common senior-level traps

"CPU is high, so add instances." Scaling may mitigate, but retry storms and shared downstream bottlenecks can worsen.

"The runnable thread in the dump is the culprit." Match OS thread CPU and repeated samples; runnable is not proof of consumption.

"The widest flame-graph frame is bad code." It may represent legitimate work; compare with workload and healthy baselines.

"Memory is stable, so allocation is fine." Short-lived allocation can create high CPU without heap growth.

"Virtual threads solve thread-related CPU." They reduce blocking-thread cost, not compute demand.

"Restarting fixed it." Restarting removed the symptom and the evidence; recurrence remains unexplained.

A strong interview answer

I first confirm CPU ownership and units across host, container, process, and threads, and check throttling, throughput, latency, GC, and recent changes. I protect the service while preserving evidence: repeated thread dumps, a bounded JFR or sampling profile, GC state, and traffic context. I map hot native threads to Java stacks or use a CPU flame graph, then correlate hot paths with endpoints, payloads, tenants, retries, allocation, and deployment version. I distinguish useful demand from wasted work such as algorithmic amplification, retry storms, exception logging, excessive allocation, GC pressure, spinning, or cold JIT activity. Mitigation is reversible and root-cause correction is validated by CPU per successful operation, latency, allocation, GC, and correctness under representative load.

Chapter summary

High-CPU diagnosis is an evidence-correlation problem. Operating-system data establishes who consumed CPU; JVM recordings establish which stacks consumed it; business metrics explain why those stacks became hot. The goal is not merely lower utilization, but more correct completed work per unit of CPU with predictable latency.

Revision Notes

- Confirm CPU units, host scope, container limits, and process ownership.
- High CPU can represent useful throughput or wasted work.
- Preserve thread dumps, a bounded profile, GC state, and workload context before restart.
- Repeated thread dumps are more useful than one snapshot.
- Match operating-system thread IDs to Java `nid` values carefully.
- `RUNNABLE` does not prove continuous CPU consumption.
- CPU profiles measure execution; wall profiles include waiting.
- Flame-graph width represents sample frequency, not blame.
- Compare profiles with a healthy baseline under similar traffic.
- Check CPU per successful operation, not utilization alone.
- Algorithmic amplification often appears only at production data sizes.
- Stable heap usage does not rule out expensive short-lived allocation.
- Separate application-thread CPU from garbage-collector CPU.
- Retry and exception storms multiply CPU and allocation.
- Bound input size for parsing, regex, compression, and serialization.
- Busy waiting and failed CAS loops can consume CPU without progress.
- Cold instances spend CPU on class loading, JIT, caches, and connections.
- Virtual threads do not increase compute capacity.
- Mitigate reversibly, preserve evidence, and verify the permanent fix under load.